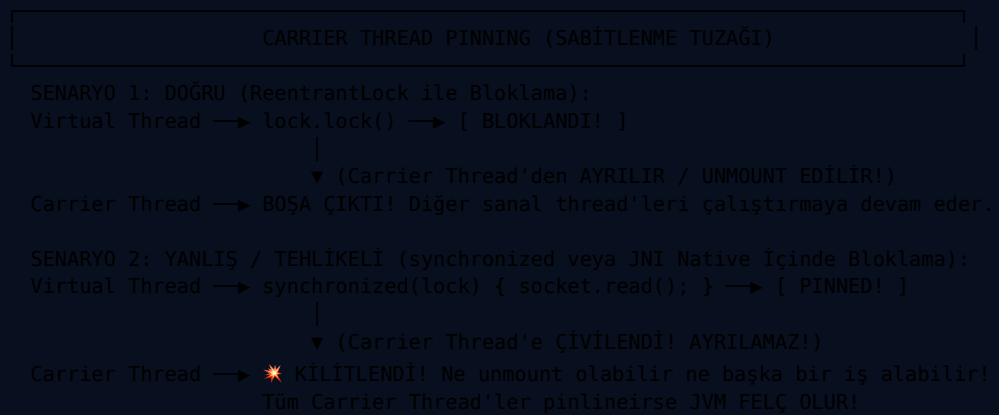
45

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

The Pinning Hazard (Sabitlenme Tehlikesi): synchronized vs ReentrantLock & Native Kod Tuzakları

The Pinning Hazard: synchronized vs ReentrantLock & Native Methods

⚡ Bellek & Eşzamanlılık Akış Şeması ⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Sanal thread bir posta güvercini gibidir; mektubu bırakıp hemen uçar. Ancak güvercinin ayağına ağır bir demir gülle bağlarsan uçamaz. 'Pinning' tam olarak budur: Eğer sanal thread `synchronized` bloğu veya eski C kütüphanesi (JNI) içindeyken ağdan veri beklerse, altındaki devasa platform thread'e çivilenir! Platform thread kımıldayamaz hale gelir. 16 çekirdeğin varsa ve 16 sanal thread pinlineirse bütün sunucu kilitlenir!

🇬🇧 ENGLISH (ELI5)

A virtual thread is an agile gymnast capable of leaping off the balance beam at any moment. Carrier Pinning is like super-gluing the gymnast's feet to the beam. If a virtual thread enters a blocking I/O operation inside a legacy `synchronized` block or a native JNI C-library call, it cannot unmount! The underlying OS carrier thread is held hostage. If all carrier threads become pinned, the entire JVM throughput collapses to zero!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Pinning Mekanizması ve Çözümler: 1) Neden Sabitlenir (Pinning)? Bir thread native C frame'i (`JNI`) yürütüyorsa veya bir nesnenin monitörüne (`synchronized`) sahipken bloklayan bir işlem yaparsa, HotSpot C++ çalışma zamanı monitör sahipliğini native stack'ten ayıramaz. Buna `Pinned Carrier Thread` denir. 2) Sonuçları: Carrier thread unmount edilemez; diğer binlerce sanal thread kuyrukta açlıktan ölür (starvation). HotSpot telafi etmek için geçici olarak `ForkJoinPool`'a ek carrier thread ekler, ancak bu OS thread patlamasına yol açar. 3) Teşhis: JVM `-Djdk.tracePinnedThreads=full` veya `short` parametresiyle başlatılarak pinlineen yerler loglanır. 4) Çözüm: Bloklayan I/O içeren tüm `synchronized` blokları derhal `java.util.concurrent.locks.ReentrantLock` ile değiştirilmelidir. (Not: Java 24/25'te synchronized pinning'i büyük ölçüde hafifletilmiştir ancak native JNI ve kritik bölümler için ReentrantLock endüstri standardıdır).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Pinning Internals & Diagnostic Protocol: 1) Root Cause: A virtual thread cannot be unmounted when its call frame contains native frames (JNI) or when holding an object monitor inside a `synchronized` block/method that enters `park()`. The HotSpot native frame metadata is bound to the carrier's OS stack. 2) Catastrophic Degradation: With all carrier threads pinned, the `ForkJoinPool` cannot service pending virtual threads. HotSpot attempts compensatory thread injection, spiking OS kernel thread counts and memory consumption. 3) Diagnostics: Enable JVM flag `-Djdk.tracePinnedThreads=full` to print stack traces whenever pinning occurs. 4) Modern Remediation: Replace blocking `synchronized` sections with `ReentrantLock`, which relies on `LockSupport.park()` and fully unmounts virtual threads without pinning.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: Pinning Tuzağı ve ReentrantLock ile Güvenli Dönüşüm
import java.util.concurrent.Executors;
import java.util.concurrent.locks.ReentrantLock;

public class PinningPreventionDemo {
    private final ReentrantLock modernLock = new ReentrantLock();
    private final Object legacyLock = new Object();

    // ❌ TEHLİKELİ ESKİ YÖNTEM: synchronized içinde I/O (Pinning Yaratır!)
    public void hazardousMethod() {
        synchronized (legacyLock) {
            try {
                // Pinning alarmı! Carrier thread unmount olamaz!
                Thread.sleep(100);
            } catch (InterruptedException _) {}
        }
    }

    // ✅ GÜVENLİ MODERN YÖNTEM: ReentrantLock (Carrier Thread Serbest Kalır!)
    public void resilientMethod() {
        modernLock.lock();
        try {
            // Virtual thread unmount olur, carrier thread serbestçe diğer işleri yapar!
            Thread.sleep(100);
        } catch (InterruptedException _) {
            Thread.currentThread().interrupt();
        } finally {
            modernLock.unlock();
        }
    }

    void main() {
        System.setProperty("jdk.tracePinnedThreads", "short");
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 5; i++) {
                executor.submit(this::resilientMethod);
            }
        }
        System.out.println("Tüm sanal thread'ler sıfır pinning ile başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski JDBC sürücüleri ve üçüncü parti kütüphanelerin derinliklerinde socket okumaları synchronized blokları içine sarılmış olabilir! Projenizi Sanal Thread'lere geçirirken kesinlikle `-Djdk.tracePinnedThreads=full` bayrağını açıp harici kütüphanelerdeki pinning noktalarını denetleyin.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy JDBC drivers, XML parsers, and HTTP clients often wrap socket reads in `synchronized` blocks. When migrating production services to Virtual Threads, always audit dependencies with `-Djdk.tracePinnedThreads=full` to eliminate stealth carrier thread starvation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• synchronized veya JNI içinde I/O yapmak Carrier Thread'i kilitler (Pinning). Bloklayan I/O bölgelerinde daima ReentrantLock kullanın.

🎯 Key Takeaways for the Interview

• Blocking I/O inside `synchronized` or JNI pins the carrier thread. Replace `synchronized` blocks wrapping I/O with `ReentrantLock`.